

# Customer Onboarding AI

## Task 6 — Domain Runtime / FastAPI Boundary

---

### Beginner-Friendly • Enterprise-Grade Implementation Guide

This guide explains Task 6 as the next incremental step after the controlled React renderer and runnable React UI. It preserves the exact Claude Code implementation prompt used for Task 6 and explains the resulting architecture, responsibilities, controls, testing, and enterprise engineering lessons.

### 1. Where Task 6 Fits

Tasks 1–5 established the business problem, project foundation, model contract, JSON Schema, deterministic validator, controlled renderer, and runnable React UI. Task 6 introduces the Domain Runtime without introducing AI.

Task 5  
Developer-authored approved UISpec

↓  
Controlled Renderer  
↓  
Browser

Task 6  
Browser / React  
↓  
HTTP API  
↓  
FastAPI Domain Runtime  
↓  
Onboarding Service  
↓  
UISpec Validator  
↓  
Approved UISpec  
↓  
Controlled Renderer  
↓  
Browser

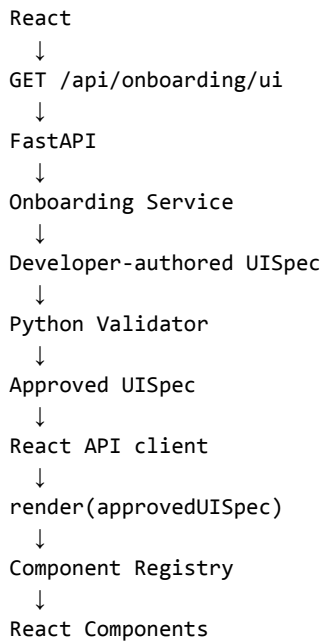
### 2. Task 6 Objective

Establish the first real runtime/API boundary between React and Python. FastAPI becomes the Domain Runtime. For this increment it returns a developer-authored UISpec only after the existing deterministic validator accepts it.

The AI Runtime remains unimplemented: no model API call, model gateway, prompt construction, authentication, database, document processing, risk scoring, or approval workflow.

### 3. The Core Architectural Rule

The React application no longer owns the runtime UISpec source. The Domain Runtime API supplies it. The controlled renderer remains the only path from an approved UISpec to React components.



### 4. What Is the Domain Service?

The domain/application service is the small orchestration point between the FastAPI route, the UISpec source, and the deterministic validator.

api/services/onboarding.py

```
get_landing_ui_spec()
    |
    |— obtain UISpec fixture
    |— validate UISpec
    |— return approved UISpec
```

It does not contain React code, HTTP response construction, model SDK calls, prompt construction, database access, or authentication. The existing domain/validator.py remains responsible for deterministic UISpec validation.

### 5. FastAPI Structure

```
api/
├── __init__.py
├── errors.py
├── main.py
├── routes/
│   ├── __init__.py
│   ├── health.py
│   └── onboarding.py
├── services/
│   ├── __init__.py
│   └── onboarding.py
```

```
└─ fixtures/
   └─ __init__.py
   └─ onboarding_ui.py
```

- `api/main.py` — creates FastAPI, configures scoped local-development CORS, and wires routers.
- `api/routes/health.py` — exposes GET `/health`.
- `api/routes/onboarding.py` — exposes GET `/api/onboarding/ui` and translates service errors.
- `api/services/onboarding.py` — orchestrates fixture retrieval and deterministic validation.
- `api/fixtures/onboarding_ui.py` — contains the developer-authored `ContextPayload` and `UISpec`.
- `api/errors.py` — defines the invalid-fixture application error.

## 6. API Contract

GET `/health`

```
200
{
  "status": "ok"
}
```

GET `/api/onboarding/ui`

```
200
<UISpec JSON>
```

The response body is the `UISpec` itself.  
No `{ "data": ... }` or `{ "ui": ... }` envelope.

If the fixture fails validation, the service does not return it. The route returns HTTP 500 with a small structured error.

```
{
  "error": "invalid_ui_spec",
  "detail": ["..."]
}
```

## 7. Developer-Authored UISpec Fixture

Task 6 needs a deterministic `UISpec` source before AI exists. The fixture is deliberately developer-authored.

`api/fixtures/onboarding_ui.py`

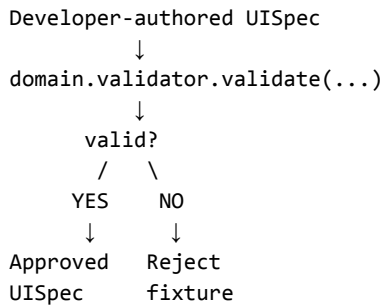
```
LANDING_CONTEXT
    segment = individual
    product_interest = checking
    region = US
```

```
LANDING_UI_SPEC
    Hero
        └─ CTA
        └─ Text
```

This temporary fixture proves the runtime/API seam before probabilistic AI is introduced.

## 8. Validator Integration

Task 6 reuses the Task 3 Python validator instead of creating a second validation system.



There is no auto-repair, substitution, or AI repair. The validator remains the deterministic trust boundary.

## 9. Why the Fixture Is Deep-Copied

The service deep-copies the module-level fixture so callers receive a separate object and cannot accidentally mutate the source fixture. Tests verify non-mutation.

## 10. Late-Bound Fixture Reference

An early implementation imported the fixture constant directly, which captured its reference at import time and prevented the invalid-fixture test from replacing it. The service was changed to access the fixture module at call time.

```
from api.fixtures import onboarding_ui as _fixture

# Read _fixture.LANDING_UI_SPEC when the service is called.
```

This improves testability and future substitution. It is not itself an implemented AI gateway.

## 11. React API Integration

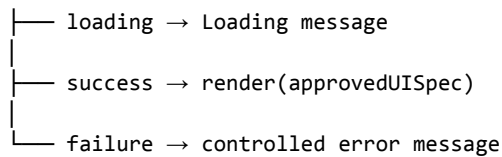
```
ui/src/api/client.ts

fetchOnboardingUiSpec()
  ↓
GET /api/onboarding/ui
  ↓
HTTP response
  ↓
basic UISpec shape guard
  ↓
UISpec returned to App
```

The client performs only an outer response-shape check; it deliberately does not recreate the complete Python validator in TypeScript.

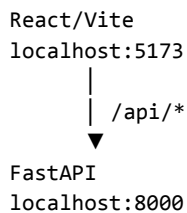
## 12. Loading, Success and Error States

```
API request
|
```



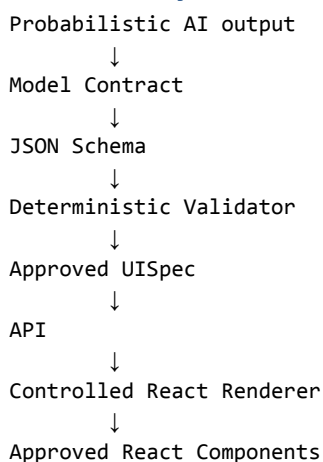
The application does not silently fall back to the retired Task 5 fixtures when the API fails. A broken runtime dependency remains visible.

### 13. Local Development Boundary



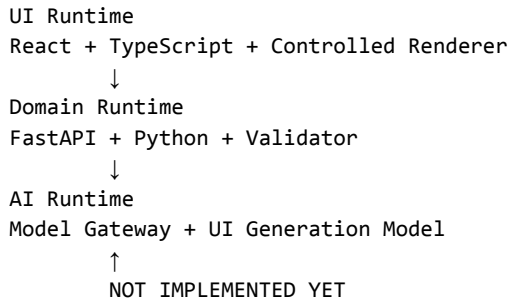
- Development origins are explicitly limited to localhost and 127.0.0.1 on ports 5173 and 4173.
- Only GET is enabled by the configured local-development CORS policy.
- Credentials are not enabled.
- Wildcard origins are not used.

### 14. Security and Control Boundary



- No eval().
- No Function().
- No dynamic component imports based on API/model data.
- No dangerouslySetInnerHTML.
- No arbitrary component resolution.
- No JavaScript URLs.
- No arbitrary HTML execution.
- No renderer bypass.
- No AI SDK is loaded.

## 15. Why There Is Still No AI Runtime



Keeping these increments separate lets us prove the deterministic runtime boundary before adding probabilistic behavior.

## 16. Testing Strategy

Task 2 → JSON Schema verification → 12/12  
Task 3 → Python validator tests → 24/24  
Task 4 → Controlled renderer → 19/19  
Task 5 → Runnable React UI → 38/38  
Task 6 → FastAPI + API/UI tests → 34/34 Python; 52/52 React

The implementation report also confirmed a successful production build and successful local API/browser smoke tests.

## 17. Task 6 Verification Results

- Python tests: 34/34 passed.
- React/Vitest tests: 52/52 passed.
- Task 2 JSON Schema verification: 12/12 passed.
- React production build: passed.
- GET /health: HTTP 200.
- GET /api/onboarding/ui: HTTP 200 with the UISpec as the response body.
- Vite /api proxy: verified end-to-end.
- CORS preflight: verified for the configured local development origin.
- Task 4 renderer security controls remained intact.
- No AI/model SDK was introduced.

## 18. Dependencies

- fastapi>=0.111
- uvicorn[standard]>=0.30
- httpx>=0.27 (FastAPI TestClient support)

No new React runtime dependency was required. No ORM, database driver, authentication library, AI SDK, agent framework, or cloud SDK was added.

## 19. What Task 6 Does Not Build

- No model gateway.

- No Claude/OpenAI/other model API integration.
- No prompt-generation layer.
- No authentication subsystem.
- No database or ORM.
- No document processing.
- No customer registration.
- No risk scoring.
- No human approval workflow.
- No production deployment/infrastructure.
- No model-contract or JSON Schema changes.
- No controlled-renderer redesign.

## 20. Exact Task 6 Claude Code Prompt

The following is the exact implementation prompt used for Task 6. It is preserved as the teaching artifact and implementation record.

Proceed with Task 6: implement the Domain Runtime / FastAPI Boundary.

This is the next smallest implementation increment for the Customer Onboarding AI application.

IMPORTANT:

Task 5 is officially completed and checkpointed.

Current verified checkpoint:

bc96e12 Task 5: Implement minimal runnable React UI

The current architecture contains:

- Business problem
- System specification
- v0 model contract
- JSON Schema
- deterministic Python UISpec validator
- controlled React component registry

- controlled UISpec renderer
- minimal runnable React UI
- developer-authored approved UISpec fixtures
- deterministic tests
- browser-runnable Vite application

Task 6 must establish the first real runtime/API boundary between the React UI and the Python domain runtime.

Do NOT start Task 7.

Do NOT implement a model gateway.

Do NOT call an AI/model API.

Do NOT implement runtime AI generation.

Do NOT introduce authentication.

Do NOT introduce a database.

Do NOT introduce document processing.

Do NOT introduce customer registration.

Do NOT introduce risk scoring.

Do NOT introduce human approval workflows.

Do NOT introduce production infrastructure.

Do NOT redesign the existing UISpec model contract.

Do NOT redesign the controlled React renderer.

Do NOT duplicate the entire Python validator in TypeScript.

=====

ARCHITECTURAL OBJECTIVE

=====

Move the application from:

Developer-authored UISpec



React Renderer



Browser

to:

React Browser UI



HTTP API



FastAPI Domain Runtime



Approved UISpec



React Controlled Renderer

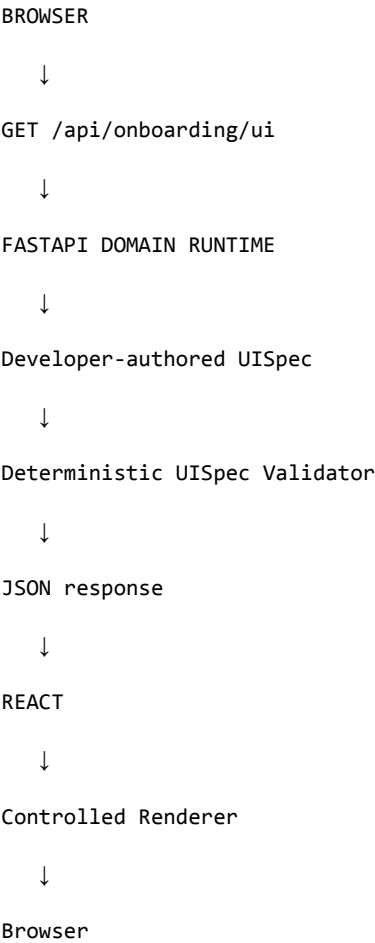


Browser UI

For Task 6, the FastAPI service should return a developer-authored approved UISpec.

There must still be NO AI/model call.

Conceptually:



The AI Runtime remains unimplemented.

=====

BEFORE IMPLEMENTATION

=====

First inspect the current repository.

Confirm the latest checkpoint and current working tree.

Read:

- CLAUDE.md
- README.md
- docs/business-problem.md
- docs/architecture.md
- specs/spec.md
- specs/model-contracts.md
- specs/schemas/ui-spec.schema.json
- domain/validator.py
- tests/domain/test\_validator.py
- ui/src/types.ts
- ui/src/registry.ts
- ui/src/render.tsx
- ui/src/App.tsx
- ui/src/fixtures/customer-onboarding.ts
- ui/package.json
- ui/README.md

Review the Task 1-5 implementation before making changes.

Before writing files, explain:

1. The FastAPI application structure you intend to create.
2. Where the domain service will live.
3. Where the developer-authored UISpec will live.
4. How the existing Python validator will be used.

5. What the API contract will be.
6. How React will consume the API.
7. How the existing controlled renderer remains the only rendering path.
8. What is intentionally NOT being implemented.

Do not proceed to implementation until this design is clear.

```
=====
FASTAPI APPLICATION
=====
```

Create the minimum FastAPI application necessary.

Use a structure similar to:

```
domain/
    validator.py
    ...

api/
    __init__.py
    main.py
    routes/
        __init__.py
        onboarding.py
    services/
        __init__.py
        onboarding.py
    fixtures/
        __init__.py
        onboarding_ui.py
```

You may adjust the exact structure if the existing repository suggests a cleaner minimal design.

Do not over-engineer the service.

=====

HEALTH ENDPOINT

=====

Create:

GET /health

It should return a small deterministic response indicating that the service is running.

For example:

```
{  
  "status": "ok"  
}
```

Do not include unnecessary infrastructure information.

=====

ONBOARDING UI ENDPOINT

=====

Create:

GET /api/onboarding/ui

The endpoint must return a valid UISpec according to the existing v0 contract.

Do not invent a new UI contract.

Use the existing:

specs/model-contracts.md

and:

specs/schemas/ui-spec.schema.json

as the source of truth.

The response must contain an approved UISpec.

=====

DOMAIN SERVICE

=====

Do not place all business logic directly inside the FastAPI route.

Create a small onboarding domain/service function responsible for obtaining the UISpec.

For this task the service may return a developer-authored fixture.

Conceptually:

FastAPI route

↓

Onboarding service

↓

Developer-authored UISpec

↓

Existing Python validator

↓

Return approved UISpec

Keep this deliberately small.

Do not introduce repositories, dependency injection frameworks, databases, or other abstractions unless genuinely required.

```
=====
PYTHON VALIDATION BOUNDARY
=====
```

The existing deterministic Python validator from Task 3 must be reused.

Do NOT duplicate validation logic.

Before the API returns the developer-authored UISpec:

1. Validate its JSON-schema structure.
2. Validate the existing runtime/cross-field rules.
3. Confirm the UISpec is accepted.
4. Return the approved UISpec.

If the developer-authored fixture is invalid, the service must fail deterministically rather than returning invalid data.

Do not auto-repair the fixture.

Do not silently modify it.

Do not call an AI model to repair it.

The validator remains the deterministic trust boundary.

=====

API CONTRACT

=====

The API response should be the UISpec itself unless there is a strong existing contract reason to introduce an envelope.

Do not introduce:

```
{  
  "data": ...  
}
```

or:

```
{  
  "ui": ...  
}
```

unless the existing project specification requires it.

Keep the API contract as small as possible.

The goal is:

GET /api/onboarding/ui

↓

UISpec JSON

This keeps the Task 6 seam clear.

=====

CORS / LOCAL DEVELOPMENT

=====

The React development server and FastAPI development server will normally run on different localhost ports.

Configure only the minimum local-development CORS support required for the React application to call FastAPI.

Do not create production CORS policy yet.

Do not allow arbitrary origins.

Use an explicit development origin.

Document the selected development ports/origin.

=====

REACT INTEGRATION

=====

Modify the React application so that it obtains the UISpec from:

GET /api/onboarding/ui

instead of directly owning the Task 5 fixture as the runtime source.

The flow should become:

React App



API client



GET /api/onboarding/ui



FastAPI



UISpec



render(approvedUISpec)



Registry



React components

The existing controlled renderer must remain unchanged unless a genuine integration issue requires a minimal change.

The application must NOT:

- import the Python validator
- import the React registry to bypass the renderer
- create React elements directly from API component names
- interpret API data as executable code
- dynamically import components

- render raw HTML from the API
- use eval()
- use Function()

The renderer remains:

```
render(approvedUISpec)
```

```
=====
CLIENT-SIDE RESPONSE HANDLING
=====
```

The React API client should handle at least:

1. Loading state
2. Successful response
3. HTTP/API failure
4. Invalid/unexpected response shape

Do not recreate the complete Python validation system in TypeScript.

For this task, the server-side validator remains authoritative.

The client should still fail safely if the response is not usable.

Do not silently substitute the old fixture if the API fails.

This is important.

A failed runtime dependency should be visible as an application error rather than silently hiding the failure by reverting to hard-coded content.

=====

## UI BEHAVIOUR

=====

Keep the existing visual design simple.

Add only the minimum states required to demonstrate:

Loading



Successful API response



Rendered UI

and:

API failure



Controlled error message

Do not build a full design system.

Do not redesign the Customer Onboarding page.

Do not add additional product functionality.

=====

## TESTING

=====

Add deterministic FastAPI/domain tests.

At minimum test:

1. /health returns HTTP 200.
2. /health returns the expected deterministic status.
3. /api/onboarding/ui returns HTTP 200.
4. /api/onboarding/ui returns a UISpec with the expected version.
5. The returned UISpec passes the existing Python validator.
6. The endpoint returns the expected developer-authored fixture.
7. An invalid fixture cannot be returned as an approved UISpec.
8. The service does not mutate the fixture unexpectedly.
9. The same request produces the same response.
10. No model/API call is made.

Add React tests for:

11. Loading state.
12. Successful API response.
13. API failure state.
14. The successful response reaches the existing controlled renderer.
15. The React application does not bypass the renderer.

Keep tests readable and beginner-friendly.

=====

DEPENDENCIES

=====

Use the minimum dependencies.

FastAPI and its required development server dependency may be added if not already present.

Do not add:

- database frameworks
- ORM
- authentication libraries
- AI SDKs
- model SDKs
- LangChain
- agent frameworks
- message brokers
- cloud SDKs

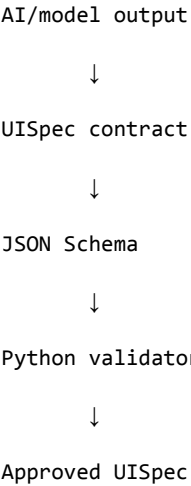
For the React side, reuse the existing dependencies wherever possible.

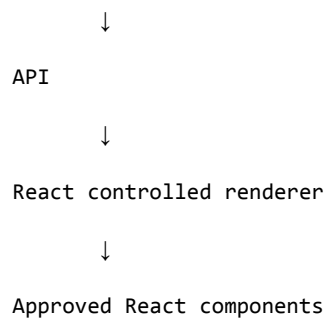
=====

SECURITY / CONTROL REQUIREMENTS

=====

Maintain the established control boundary:





Even though Task 6 does not yet contain an AI model, preserve this future architecture.

The API must not become an arbitrary component execution mechanism.

Do not allow:

- arbitrary component names to cause dynamic imports
- arbitrary HTML
- JavaScript execution
- shell execution
- arbitrary URL execution
- arbitrary DOM properties

=====

IMPORTANT ARCHITECTURAL DISTINCTION

=====

Do not describe the FastAPI service as the "AI Runtime".

FastAPI is the Domain Runtime.

The architecture remains:

UI Runtime

- React
- TypeScript
- Controlled GenUI renderer

#### Domain Runtime

- FastAPI
- Python
- Business/API logic
- UISpec validation

#### AI Runtime

- Model gateway
- UI generation model

The AI Runtime remains unimplemented during Task 6.

=====

#### DOCUMENTATION

=====

Update the relevant project documentation.

At minimum:

- ui/README.md
- README.md if required
- tests/README.md if required

If the architecture documentation needs a small update to reflect the newly implemented Domain Runtime, update it carefully without rewriting the architecture.

Document:

- how to install FastAPI dependencies
- how to start the FastAPI server
- how to start the React development server
- expected local ports
- GET /health
- GET /api/onboarding/ui
- the runtime flow
- the fact that the API currently returns a developer-authored UISpec
- the fact that AI/model integration is intentionally not implemented

Do not update the walkthrough yet unless specifically required.

The walkthrough will be updated separately after the checkpoint.

=====

#### VERIFICATION

=====

After implementation, run:

1. FastAPI/domain tests.
2. React tests.
3. Existing Task 4 renderer tests.
4. Existing Task 3 validator tests.
5. Existing Task 2 schema verification.
6. React production build.
7. Start FastAPI locally.
8. Verify GET /health.
9. Verify GET /api/onboarding/ui.

10. Start React locally.
11. Verify the browser application loads data from FastAPI.
12. Verify API failure state.
13. Review git diff.

Confirm:

- no AI/model integration
- no model gateway
- no authentication
- no database
- no document processing
- no risk functionality
- no approval workflow
- no production infrastructure
- no renderer bypass
- no duplicate Python validator
- no arbitrary component execution

=====

GIT DISCIPLINE

=====

Do NOT commit.

Do NOT push.

Do NOT start Task 7.

After all verification is complete, report:

1. Files created
2. Files modified
3. FastAPI architecture
4. API endpoints
5. Domain service design
6. UISpec fixture location
7. Validator integration
8. React API integration
9. Loading/error behaviour
10. Tests added
11. Full test results
12. Schema regression results
13. Build results
14. Browser/API smoke-test results
15. Dependencies added
16. Security/control checks
17. Documentation changes
18. Any architectural issues discovered
19. Git diff summary
20. Recommended next smallest task

STOP after the report.

Wait for human review before proceeding to Task 7.

## 21. Task 6 Implementation Results

The implementation report confirmed that the requested Domain Runtime boundary was established without AI integration. The implementation contains the API layer, onboarding service, fixture, validator integration, React API client, loading/error states, tests, and local development wiring.

- 34/34 Python tests passed.
- 52/52 React/Vitest tests passed.
- 12/12 JSON Schema checks passed.
- Production build passed.
- GET /health returned 200.

- GET /api/onboarding/ui returned 200 with the UISpec as the body.
- Vite API proxy was verified end-to-end.
- CORS preflight was verified for the configured local origin.
- Task 4 security controls remained intact.
- No AI/model SDK was introduced.

## 22. Enterprise Engineering Lessons

- Keep HTTP concerns in routes and application/domain orchestration in services.
- Reuse deterministic validators rather than duplicating validation logic.
- Make runtime failures visible; do not silently fall back to stale fixtures.
- Keep API contracts deliberately small until a real requirement expands them.
- Separate UI Runtime, Domain Runtime, and AI Runtime.
- Introduce probabilistic AI only after deterministic runtime boundaries are working.
- Use tests at each trust boundary.
- Checkpoint each verified increment so the project remains recoverable.

## 23. What Comes Next

Task 6

FastAPI Domain Runtime



Next small seam

ContextPayload request



Future

Model Gateway



Runtime Model Integration



AI Evaluation



Feedback / Continuous Improvement

The next increment should remain small enough to verify independently. In particular, the existing model contract already defines ContextPayload, while the current API does not yet accept runtime context. This makes a ContextPayload request seam a natural candidate before the model gateway.

## 24. Key Mental Model

AI proposes



Contract constrains



Schema validates structure



Validator checks runtime rules



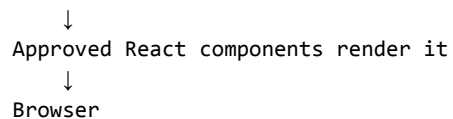
Domain Runtime serves approved UISpec



React API client receives it



Controlled Renderer maps it



## 25. Task 6 Summary

Task 6 turns the local React demonstration into a small runtime-backed application. FastAPI now acts as the Domain Runtime, the onboarding service owns orchestration logic, the existing Python validator remains the deterministic trust boundary, and React consumes the UISpec through an API before passing it to the controlled renderer. AI remains deliberately outside the implementation.